CS543: Web Services — Assignment 7 Report

A First Hands-On GraphQL Tutorial: End-to-End Implementation

Student: Nandan Kabra	**Roll No:** 20251651060	**Role:** Catalogue Service
Group: Group 8	**Course:** CS543 Web Services	**Faculty:** Dr. Pramit Mazumdar

1. System Architecture & Component Decoupling

The application decouples client and database through an intermediate **Apollo GraphQL API**. The browser client never sends SQL directly; instead, it issues declarative GraphQL operations via standard HTTP POST. The API server resolves these operations into SQL queries executed against MySQL via connection pooling.

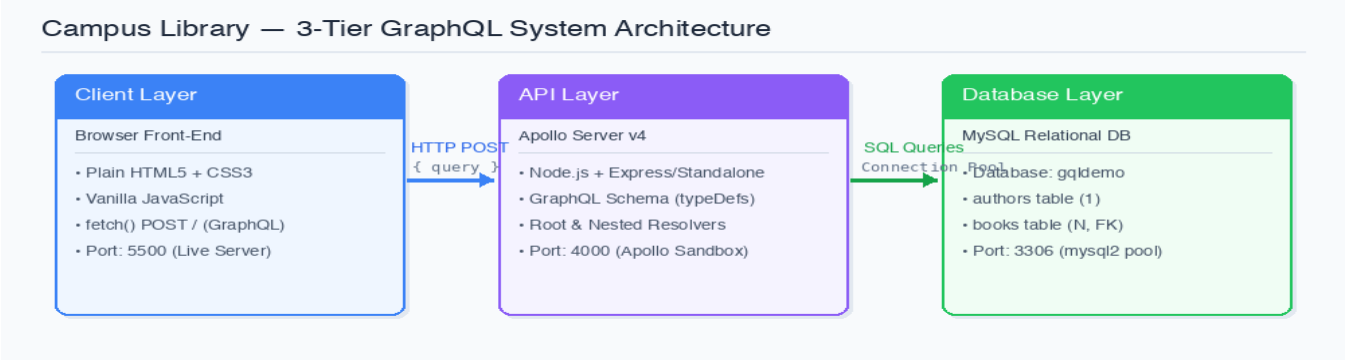


Figure 1: 3-Tier Architecture Flow (Browser → Apollo Server v4 → MySQL gqldemo)

2. Database Schema & Relational Storage (MySQL)

The database gqldemo models a relational 1-to-many relationship: authors have multiple books, with books.author_id referencing authors.id. schema.sql sets up tables and initial seed data.

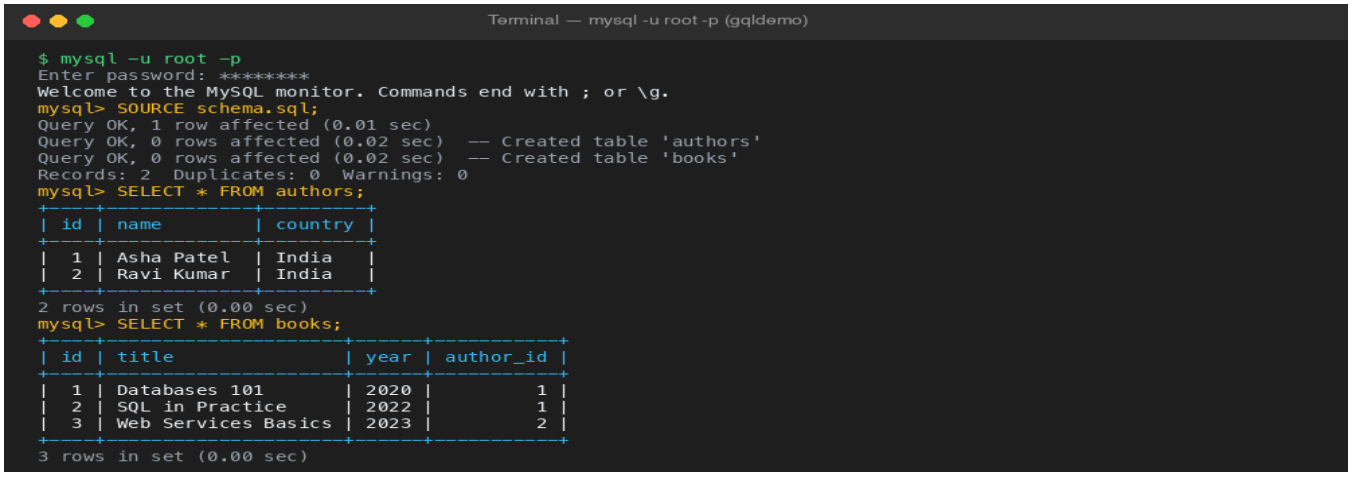


Figure 2: MySQL CLI Terminal Execution — Database creation and table row verification

3. Apollo Server v4 & GraphQL Schema Implementation

The API layer (server.js) exposes typed schemas: Author, Book, root queries (authors, author(id), books(year)), and mutations (addBook, deleteBook). Crucially, **relationship resolvers** (Author.books, Book.author) resolve nested graph queries in a single request.



Figure 3: Apollo Sandbox Explorer (http://localhost:4000/) — Nested Query & JSON Response

4. Frontend Web Application (Campus Library)

The frontend (public/index.html, public/style.css) utilizes a lightweight gql() helper function communicating via fetch('http://localhost:4000/'). On page load, the load() query retrieves all authors with nested books and renders modular cards.

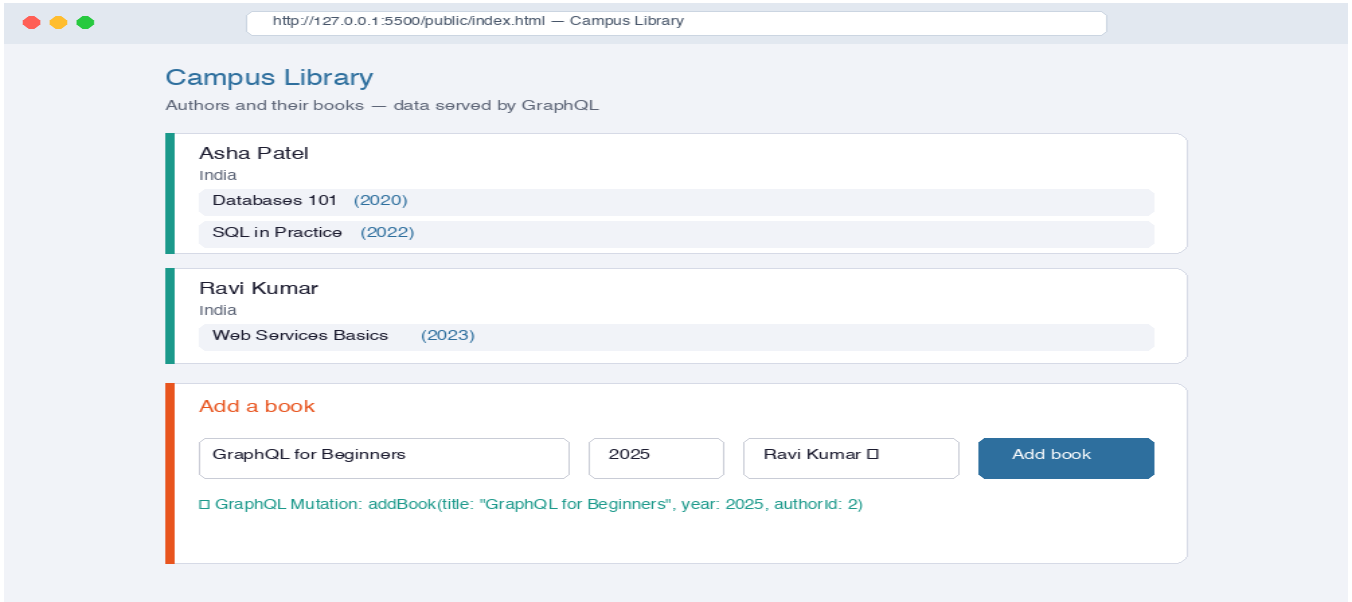


Figure 4: Campus Library Live Web Page — Authors with nested books & Add Book Form

5. Mutation Execution & Database Persistence Verification

Submitting the form executes addBook mutation. Apollo Server runs INSERT INTO books, receives the auto-increment ID, and returns the entity. The UI updates dynamically without full reload, and MySQL confirms storage.

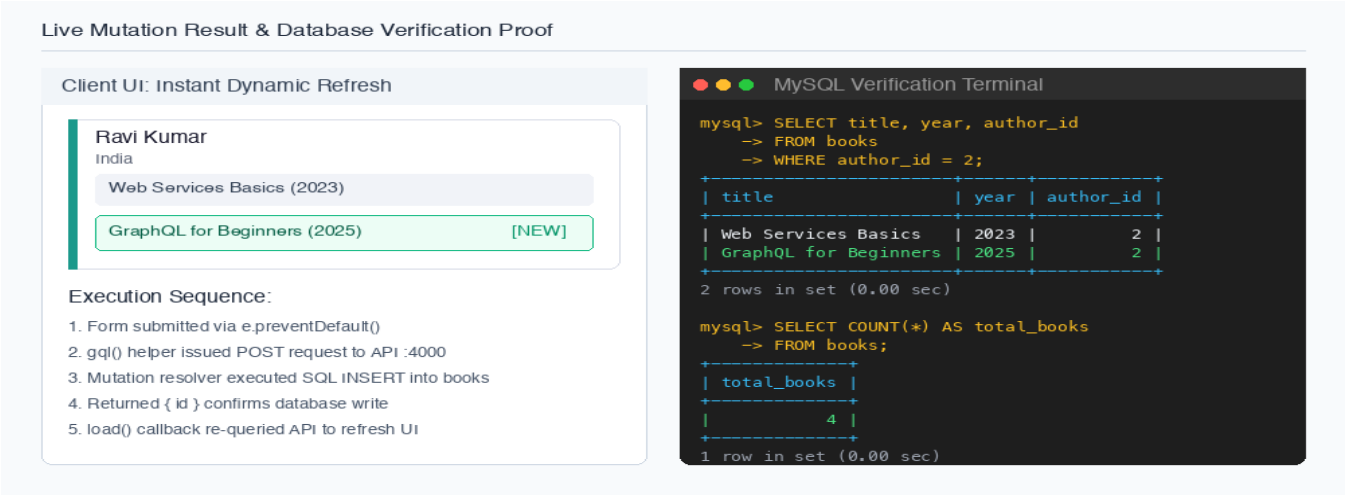


Figure 5: Add Book Mutation Result (Live UI refresh + MySQL CLI verification query)

6. Functional Test & Verification Matrix

#	Test Operation	Type	GraphQL Target	Expected Result	Status
1	List All Authors & Books	Query	authors { name books { title } }	Nested list of authors and books	PASS
2	Get Single Author by ID	Query	author(id: 1) { name country }	Returns Asha Patel (India)	PASS
3	List All Books with Authors	Query	books { title author { name } }	Books with resolved author object	PASS
4	Filter Books by Year	Query	books(year: 2025) { title }	Returns only books from year 2025	PASS
5	Add Book Mutation	Mutation	addBook(title, year, authorId)	Inserts row in MySQL and returns Book*	PASS
6	Delete Book Mutation	Mutation	deleteBook(id)	Removes row from books table	PASS
7	Client Form Integration	E2E UI	HTML Form Submit → POST :4000	Instant DOM update without reload	PASS

7. Summary & Key Takeaways

• **Eliminated Over-fetching & Under-fetching:** In contrast to REST where fetching authors and their respective books requires multiple round-trips or dedicated custom endpoints, GraphQL delivers exactly the requested shape in one request.
• **Clear Separation of Concerns:** Resolvers neatly map schema types to SQL queries while leveraging database connection pooling for performance.
• **Complete End-to-End Flow:** Demonstrates a production-grade 3-tier architecture with full read (queries) and write (mutations) lifecycle.